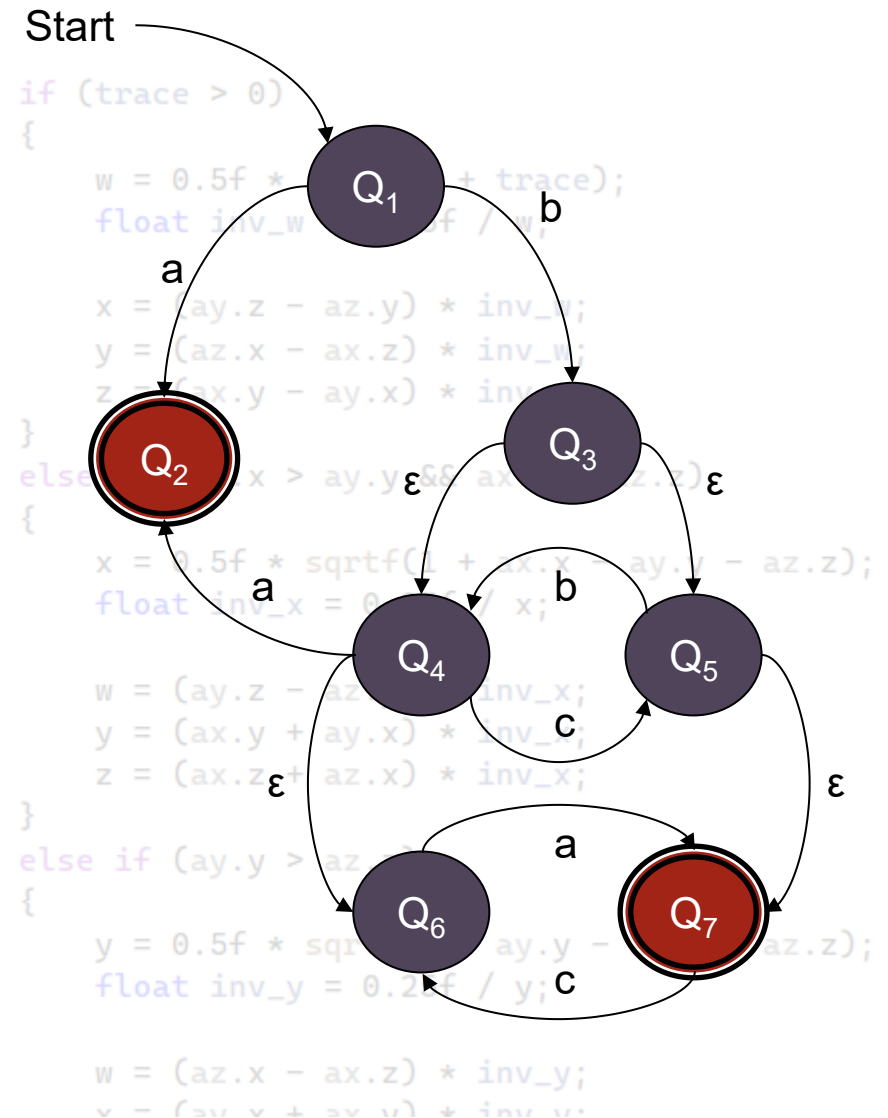


Algorithmen und Datenstrukturen

5 – Container

Tobias Werner M. Sc.
Uni Bayreuth - Angewandte Informatik 3
Leibniz-FH Hannover
Duale Hochschule Sachsen
tobias.werner@uni-bayreuth.de



Einleitung

Heranführung

In Ihrem Job als Programmierer bei einem Autokonzern bekommen Sie die Aufgabe, ...

- ... umfangreiche Echtzeit-Daten von einem Sensor aufzuzeichnen.
- ... eine Warteschlange für abzuarbeitende Fahrassistenz-Ereignisse zu realisieren.
- ... Duplikate aus einer großen Menge an Seriennummern zu filtern.

Wie gehen Sie dabei jeweils vor?

Kompetenzen

- Begriffe im Zusammenhang mit Containern erlernen
- den richtigen Container anhand von Kriterien auswählen
- typische Funktionen auf Containern kennen

5 – Container

- Begriffe und Eigenschaften
- Iteratoren und Allokatoren
- STL-Container
- Member von STL-Containern

Begriffe im Container-Umfeld (1)

Was ist ein Container?

verschiedene Definitionen, z. B. allgemein:

"Ein *Container* hält den Besitz über eine Sammlung untergeordneter Objekte und definiert, wie diese Objekte angeordnet, traversiert und abgerufen werden."

praktische Interpretation:

- Sammlung von Objekten
 - 0 bis (fast) beliebig viele?
 - Welchen Typ haben die Objekte?
- Anordnung der Objekte
 - z. B. als Liste oder als Baum
- Anordnung und Implementierung trennen
 - z. B. Baum als AVL-Baum oder RB-Baum
 - C++-Standard schreibt keine Impl. vor!
- Traversieren der Objekte
 - z. B. per Zeiger, Index oder Iterator
- Abrufen der Objekte
 - z. B. Methoden zum Zugriff mit Index

Begriffe im Container-Umfeld (2)

Begrifflichkeiten

Objekt in einem Container

→ alternative Begriffe:

- ein **Element** (im Container)
- ein *Eintrag* (im Container)
- ein *Wert* (im Container)
- C++-Dokumentation: `element`
- C++-Bezeichner: `t`, `value`, `v`

Objekttyp innerhalb des Containers

→ alternative Begriffe:

- der **Elementtyp** (des Containers)
- der *Eintragstyp* (des Containers)
- der *Werttyp* (des Containers)
- C++-Dokumentation: `element type`
- C++-Bezeichner: `τ`, `value_type`

unterscheiden zwischen:

- *Containertyp* → `std::list<int>`
- *Elementtyp* → `int` bei `std::list<int>`

- *Containervorlage* → `std::list`
- *Containerklasse* (= *Containervorlageninstanz*) → `std::list<int>`
- *Containerinstanz* → `x` bei `std::list<int> x;`
→ mit Container ist (wechselnd) einer der vier Begriffe gemeint

Eigenschaften im Container-Umfeld (1)

Typische Container-Operationen

Container-Operation ist Methode von bzw. Funktion auf Container

Erzeugen und Befüllen

- Container erzeugen → (Default-)Konstruktor
- Container kopieren → z. B. Copy-Konstruktor, Zuweisungsoperator
- Objekt einfügen → z. B. `insert()`-Funktion

Abfragen und Bearbeiten

- Container traversieren → z. B. nativer C-Zeiger und Zeigerarithmetik
- auf ein Objekt zugreifen → z. B. `operator [ ](size_t index)`
- Zahl der Objekte abfragen → z. B. `size()`-Funktion

Leeren und Löschen

- Objekt löschen → z. B. `erase()`-Funktion
- alle Objekte löschen → z. B. `clear()`-Funktion
- Container löschen → Destruktor

Eigenschaften im Container-Umfeld (2)

Eigenschaften von Operationen, Containern und Elementtypen

- **Operationseigenschaften** hängen vom Containertyp ab

Beispiele:

- Suchen Array → lineare Laufzeit, Baum → logarithmische Laufzeit
- Einfügen Array → invalidiert Referenzen, Liste → keine Invalidierung

- Operationsimplementierungen verlassen sich auf **Eigenschaften des Containers**

Beispiele:

- Elemente liegen in einer Baumstruktur vor
- Elemente lassen sich kopieren

- Container stellt **Eigenschaftsforderungen an Elementtyp**

Beispiele:

- Elementtyp hat Vergleichsoperator für Baumstruktur
- Elementtyp hat Kopierkonstruktor

→ **Aus allen Eigenschaften ergibt sich die Bewertung eines Containers!**

Eigenschaften im Container-Umfeld (3)

Eigenschaftsforderungen an Elementtypen

- Containervorlagen ermöglichen allgemeine Elementtypen

Beispiel: `std::vector<T>` erlaubt z. B. $\tau = \text{int}$ oder $\tau = \text{std::string}$

Bewertung:

- Container muss nicht für jeden Inhaltstyp neu implementiert werden...
- ... aber viele Container-Operationen hängen vom Elementtyp ab!

Beispiele:

- Wie sind Elemente zu kopieren oder zu verschieben?
- Wie sind Elemente in einem Suchbaum zu sortieren?

Folge:

- Containervorlagen fordern *Konzepte* (Schnittstellen + Semantik) von Elementtyp τ

Beispiele:

- Elemente in Array-Container brauchen Copy- oder Move-Konstruktor
- Elemente in Baum-Container müssen sortierbar sein

Eigenschaften im Container-Umfeld (4)

Laufzeit und Speicherbedarf

Sei n die Zahl der Elemente in einem Container, damit:

Speicherbedarf als Container-Eigenschaft:

Wie viel Speicherplatz braucht ein Container in Abhängigkeit von n ?

Laufzeit als Funktions-Eigenschaft:

Wie lange braucht eine Operation in Abhängigkeit von n ?

Angabe als O-Notation:

- *worst-case* → Aufwand im schlimmsten Fall, übliche Angabe
- *amortisiert* → Aufwand gemittelt über alle Elemente, va. bei Bäumen
- *best-case* → Aufwand im besten Fall, eher selten

Beispiele:

- `std::list` braucht $O((c + \text{sizeof}(T)) * n)$ worst-case-Speicherplatz
- `std::vector::push_back()` hat $O(1)$ best-case-Laufzeit

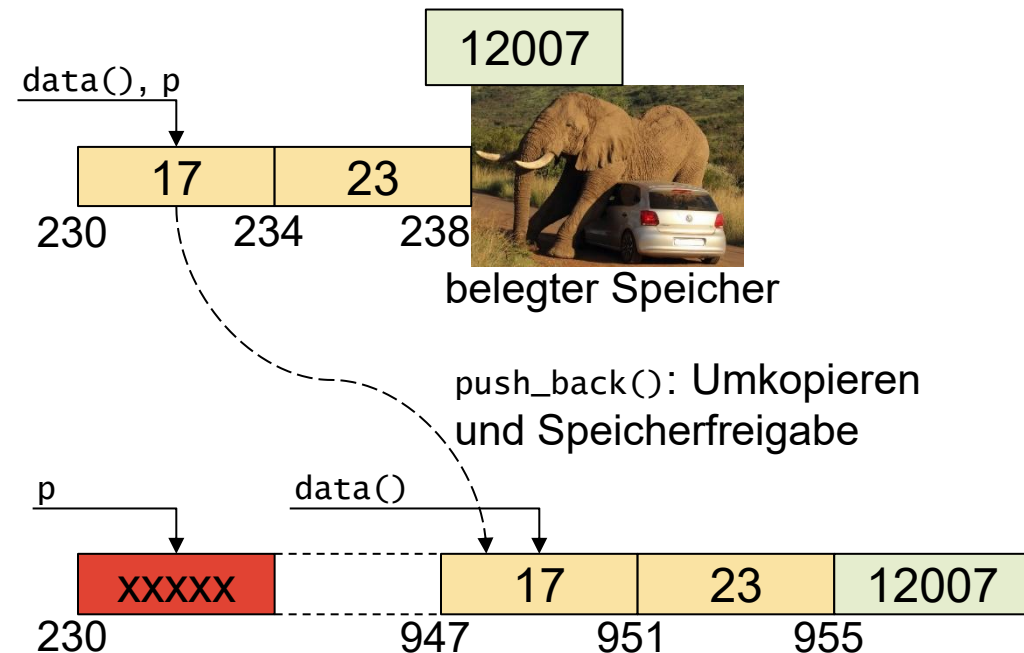
Eigenschaften im Container-Umfeld (5)

Referenz-Invalidierung als Funktionseigenschaft

- manche Container-Operationen invalidieren Referenzen, Iteratoren, Zeiger auf...
 - ... ein Objekt → z. B. `list<T>::erase()`
 - ... manche Objekte → z. B. `vector<T>::erase()`
 - ... alle Objekte → z. B. `vector<T>::push_back()`
- undefiniertes Verhalten bei invalidiertem Zugriff**
- Invalidierung ist pro Container-Funktion dokumentiert**

Beispiel:

```
std::vector<int> v( { 17, 23 } );  
int* p = v.data();  
v.push_back(12007);  
p[0] = 16;    // undef. VH!
```



<http://i1.mirror.co.uk/incoming/article3997600.ece/alternates/s2197/An-elephant-relieves-an-itch-on-a-small-car.jpg>

Eigenschaften im Container-Umfeld (6)

Fazit

There is no free lunch!

(etwa: Gratis gibts nix!)



jeder Container ist ein Kompromiss:

- z. B. `std::vector` erlaubt schnellen Zugriff per Index ABER invalidiert Iteratoren
- z. B. `std::vector` braucht keine sortierbaren Elemente ABER Suche nach Wert dauert lang
- z. B. `std::list` invalidiert keine Iteratoren ABER muss nach aufwendig nach Indizes suchen
- z. B. `std::set` sortiert die Inhalte ABER braucht zusätzlichen Speicher für die Baumknoten

Folgen für die Praxis:

Informatiker müssen...

- ... den richtigen Container für die eigene Anwendung auswählen
- ... dazu muss man die Container und ihre Eigenschaften kennen

→ **Diese Kompetenzen sind die "Mindestexpectation" an ein Informatikstudium!**

5 – Container

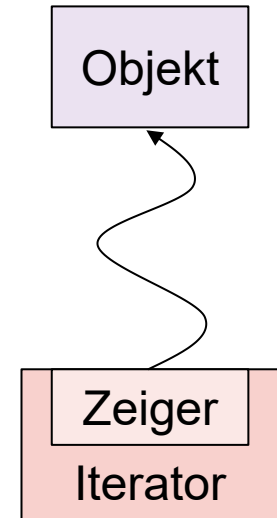
- Begriffe und Eigenschaften
- Iteratoren und Allokatoren
- STL-Container
- Member von STL-Containern

Iteratoren (1)

Grundlagen

Idee: abstrahiere den Verweis auf ein Container-Element vom Container

- Verweise werden grundsätzlich beim Durchgehen / Traversieren / Iterieren aller Element eines Containers genutzt
→ Bezeichnung **Iterator**
- Anwendung übersteigt aber das bloße Iterieren
→ z. B. auch als Marker für Lösch- oder Einfügeposition
- Verhalten von typischen Iteratoren nahe an (aber nicht gleich zu) C-Zeigern
Beispiele:
 - `operator ->` Dereferenzieren
 - `operator ++` Weiterzählen
 - `operator [ ]` Zugriff auf *n*-tes Element nach dem aktuellen Element
(das sind C++-Methoden mit dem Namen `operator [Rechenzeichen]`)



Iteratoren (2)

Praktischer Nutzen

```
template <typename C>
    void simple_sort(C& container)    // C kann list<int> oder vector<string> sein
{
    for (typename C::iterator i(container.begin()); i != container.end(); ++i)
        for (typename C::iterator j(i); j != container.end(); ++j)
            if (*j < *i) std::swap(*i, *j);
}
```

Beobachtung:

- in vielen Algorithmen lässt sich der Container abstrahieren
(z. B.: man braucht nicht wissen, wie man von einem Element zum nächsten kommt, sofern man weiß, dass das grundsätzlich funktioniert.)

Folgerungen:

- Algorithmen lassen sich als und/oder mit Vorlagen umsetzen
- Algorithmen-Vorlagen können auf bestimmten Zielcontainern arbeiten
- Iteratoren bilden als Zugriffsweg auf Elemente die Schnittstelle

Iteratoren (3)

Beobachtungen

- bestimmte Algorithmen brauchen bestimmte Operationen auf Iteratoren
Beispiele:
 - schreibender Zugriff aufs Iteratorziel für eine Netzwerknachricht
 - schneller Zugriff auf n -tes Element für Algorithmus Bucketsort
- bestimmte Container unterstützen nicht alle Operationen auf Iteratoren
Beispiele:
 - `const`-markierter Eingabepuffer für eingehende Nachrichten
 - schneller Zugriff auf n -tes Element in Container Liste

Praxisbedeutung:

- Frage: Welche Iterator-Operationen braucht ein Algorithmus?
- Frage: Welche Iterator-Operationen unterstützt ein Container?

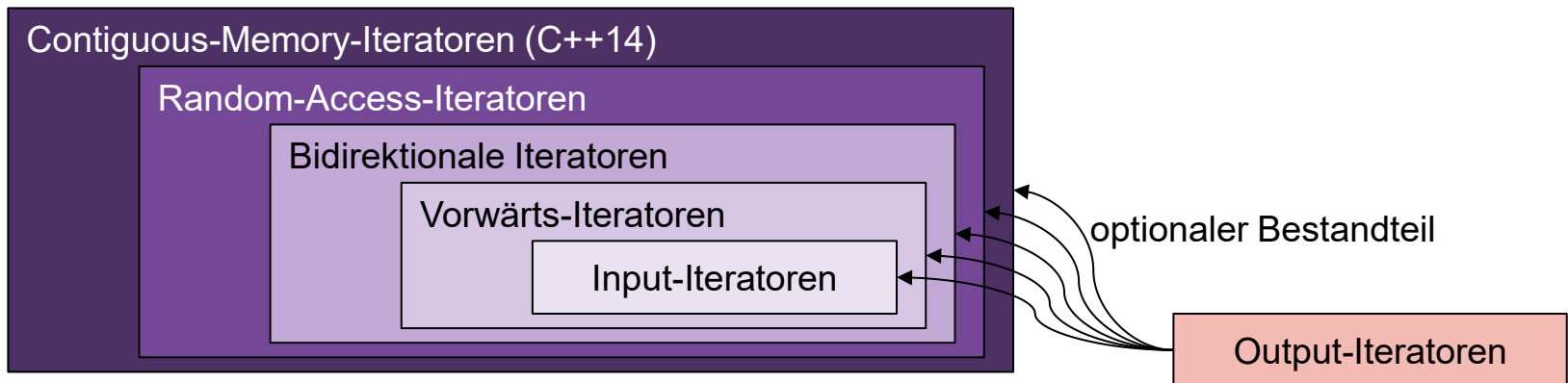
→ **Anforderungen an Iteratoren für Algorithmen und Container spezifizieren!**

Iterator-Kategorien (1)

Iteratoren als Kategorien (bis C++17)

Ansatz:

- fasse Operationen zu häufigen Iteratorkategorien (= **spezielle Konzepte!**) zusammen
- Container und Algorithmen dokumentieren angebotene oder geforderte Kategorien
- Iteratorkategorien bilden zusammen eine Hierarchie



immer enthaltene Konzepte:

- | | |
|---------------------------|--|
| • copy-konstruierbar | → <code>iterator::iterator(const iterator& other)</code> |
| • copy-zuweisbar | → <code>iterator::operator =(const iterator& other)</code> |
| • zerstörbar | → <code>iterator::~iterator() noexcept</code> |
| • um eins inkrementierbar | → <code>iterator::operator ++()</code> |

Iterator-Kategorien (2)

Input-Iteratoren

implementierte Konzepte:

- Gleichheit
 - `iterator::operator ==(const iterator& other) const`
 - `iterator::operator !=(const iterator& other) const`
- zum Lesen dereferenzierbar
 - `const target& iterator::operator *() const`
 - `const target* iterator::operator ->() const`
 - beides auch ohne `const`-Zugriff auf `target` zulässig

typische Anwendungen:

- serielles Lesen aus einer Quelle (z. B. Datei, Netzwerkpuffer)
- Invalidieren von Daten nach Iterator-Operationen (z. B. Events in einer Event-Liste)

Beispiel:

```
event_iterator it_a(first_event()), it_b(it_a); // event_iterator ist eigener Input-Iterator
process_event(*it_a);                          // * darf Ziel-Zugriff ungültig machen (1)
++it_a;                                         // ++ darf Zugriff auf Ziel ungültig machen (2)
**it_b;                                         // falls (1) oder (2) zutrifft → undef. VH
```

Iterator-Kategorien (3)

Output-Iteratoren

implementierte Konzepte:

- zum Schreiben dereferenzierbar
 - `target& iterator::operator *() const`
 - `target* iterator::operator ->() const`
 - beides nur ohne const-Zugriff auf target zulässig

typische Anwendungen:

- serielles Schreiben in ein Ziel (z. B. Datei, Netzwerkpuffer)
- Invalidieren von Daten nach Iterator-Operationen (z. B. Events in einer Event-Liste)

Beispiel:

```
message_iterator it_a(first_free_slot()); // message_iterator ist eigener Output-Iterator
message_iterator it_b(it_a);
it_a->store_and_send("Hello world!");    // -> darf Ziel-Zugriff ungültig machen (1)
++it_a;                                 // ++ darf Zugriff auf Ziel ungültig machen (2)
**it_b;                                  // falls (1) oder (2) zutrifft → undef. VH
```

Iterator-Kategorien (4)

Vorwärts-Iteratoren

implementierte Konzepte:

- Input-Iterator und **optional Output-Iterator** → jeweils mit allen Methoden
- default-konstruierbar → `iterator::iterator()`
- unabhängiges Dereferenzieren und Iterieren → nur Semantik bestehender Methoden

typische Anwendung:

- Container mehrfach durchgehen und Elemente bearbeiten (z. B. einfaches Sortieren)
- Iterator auf bestimmtes Element merken (z. B. Maximum)

Beispiel:

```
vector<int>::iterator max;                                // Default-Konstruktor nötig!
for (vector<int>::iterator i(v.begin()); i != v.end(); ++i) {
    if (max == vector<int>::iterator() || *max < *i) max = i; // Ziel von i bleibt gültig
    for (std::vector<int>::iterator j(i); j != v.end(); ++j) // hier auch
        if (*j < *i) std::swap(*i, *j);                     // und hier
}                                                            // trotz *i und ++i
```

Iterator-Kategorien (5)

Bidirektionale Iteratoren

implementierte Konzepte:

- Vorwärts-Iterator → mit allen Methoden
- dekrementierbar → `iterator::operator --()`

typische Anwendung:

- Container in beide Richtungen durchgehen (z. B. Durchhangeln durch Objektrelationen)
- beliebiger Zugriff ist nicht (effizient) möglich (z. B. Listencontainer, Datenbankcursor)

Beispiel:

```
result_iterator d(database.query("animals"));
while (d->name != "doggo") ++d;
result_iterator f(d);
if (d->best_friend > "doggo")
    while (f->name != d->best_friend) ++f;
else
    while (f->name != d->best_friend) --f;
```

// Suchen in richtige Richtung

Iterator-Kategorien (6)

Random-Access- und Contiguous-Memory-Iteratoren

implementierte Konzepte:

- bidirektionaler Iterator → mit allen Methoden
- arithmetische Operatoren → `iterator::operator +(size_t offset)`
→ `iterator::operator -(size_t offset)`
→ außerdem `+=`, `-=`
- Abstandsrechnung → `size_t iterator::operator -(const iterator& other) const`
- Vergleichsoperatoren → `bool iterator::operator <=>(const iterator& other) const`
→ außerdem `<`, `<=`, `>=`, `>`
- Zugriff mit Verschiebung → `const target& iterator::operator [] (int offset) const`
→ evtl. auch ohne `const`-Zugriff, vgl. Output-Iterator
- Array-Anforderungen → alle Einträge gleich groß und konsekutiv im Speicher
→ nur bei Contiguous-Memory-Iteratoren (C++14)

typische Anwendung:

- beliebiger Zugriff auf Container (z. B. für komplexere Sortieralgorithmen)
- alles mit Arrays

Iterator-Konzepte

Iteratoren als Konzepte (ab C++20)

Ansatz:

- ermögliche feingranulare Auswahl an Iterator-Eigenschaften
- verwende "Concepts Lite" um Iterator-Eigenschaften zur Compile-Zeit zu prüfen

Umsetzung:

- ehemalige Iterator-Kategorien als echte Konzepte
- ehemalige Hierarchie als Vererbungsbeziehung der Konzepte
- Umbenennung der alten Kategorien als Legacy*

Beispiele:

- `weakly_incremtable` → Compilezeit-Prüfung: Hat der Typ einen `operator++`?
- `output_iterator` → separat wählbares Zusatzkonzept
- `random_access_iterator` → `bidirectional_iterator` mit Prüfung für z. B. `operator [ ]`
- `LegacyInputIterator` → bezeichnet eine ungeprüfte Input-Iterator-Kategorie

→ **Details übersteigen den Vorlesungsumfang!**

Naive Speicherallokation (1)

Motivation

Praxiserfahrung:

Container von der Stange haben in bestimmten Anwendungsfällen zu viel Overhead!

Beispiel:

```
std::set<color> image_colors;           // erlaubt gleiche Elemente nur einmal
for (int y = 0; y < i.size_y(); ++y)   // finde "eindeutige" Farben in image i
    for (int x = 0; x < i.size_x(); ++x)
        image_colors.insert(i.color_at(x, y)); // O(log n) Laufzeit, das sollte gut sein!
```

Problem:

- Farbensammeln braucht für große Bilder sehr lange

Diagnose:

- Prüfung mit Profiler → großer Laufzeitbedarf bei insert
- Prüfung mit Prozessviewer → großer Speicherbedarf für Fotos

Naive Speicherallokation (2)

Diagnose

- die meisten Container erlauben 0 bis beliebig viele Elemente
- zur Umsetzung ist in der Regel dynamischer Speicher erforderlich

Beispiel: `new list_node<color>(value)` innerhalb von `image_pixels`

→ Frage: **Was macht das `new` intern?**

```
list_node<color>* new(const color& value)                                // Achtung: Pseudocode!
{
    // allokieren Speicher
    lock_mutex(global_heap_mutex);                                     // Threadsicherheit
    void* block = find_free_block(sizeof(list_node<color>));          // Blocksuche
    mark_block_used(block);                                           // Blockentnahme
    release_mutex(global_heap_mutex);

    // rufe Konstruktor auf Speicher auf
    new (block) list_node<color>(value);                               // wenig zu tun, schnell!

    return block;
}
```

Naive Speicherallokation (3)

Exkurs placement new

- `new x(...)` → **"normales"** new
- lege Speicher für x "irgendwie" auf dem Heap an (vgl. vorige Folie)
 - rufe `x(...)` auf dem angelegten Speicher auf
 - gibt Zeiger auf neu allokiertes und konstruiertes Objekt zurück
- `new (ptr) x(...)` → **"placement"** new
- verwende vorbereiteten Speicher am Ziel von `ptr` für x
 - rufe `x(...)` auf dem vorbereiteten Speicher auf
 - gibt Zeiger auf neu konstruiertes Objekt zurück

Beispiel placement new:

```
void main()
{
    char c[sizeof(database)];           // genug Speicher auf dem Stack
    void *mem = c;                      // alternativ: new(c) auch OK
    database* db = new (mem) database("db.sql"); // konstruiere db auf Stack
}
```

Naive Speicherallokation (4)

Bewertung Laufzeit new

Das gewöhnliche new ist eher langsam!

Benchmarks aktuelle PCs, Rübe nach Arm

- keine andere Last: 10.000.000 pro Sekunde
- Last, mehrere Threads: 100.000 pro Sekunde
- Debug-Modus kostet 50% bis 90% Durchsatz!

Algorithmen als Begründung:

- Suche nach freiem Block passender Größe
- Anpassung der Liste freier Blöcke

Threadsicherheit als Begründung:

- nur ein Thread darf zeitgleich in globaler Blockliste suchen und entnehmen
- andere Threads müssen bei zeitgleicher Speicherforderung warten
 - Buchführung ("bin jetzt dran", "habe fertig") ist teuer
 - Warten ("ein anderer Thread ist dran") ist extrem teuer
 - Details: siehe Betriebssysteme-VL

Naive Speicherallokation (5)

Bewertung Speicherverbrauch new

Das gewöhnliche new braucht viel zusätzlichen Speicher!

Benchmarks aktuelle PCs, Rübe nach Arm

→ 100 Byte Zusatzbedarf pro Allokation

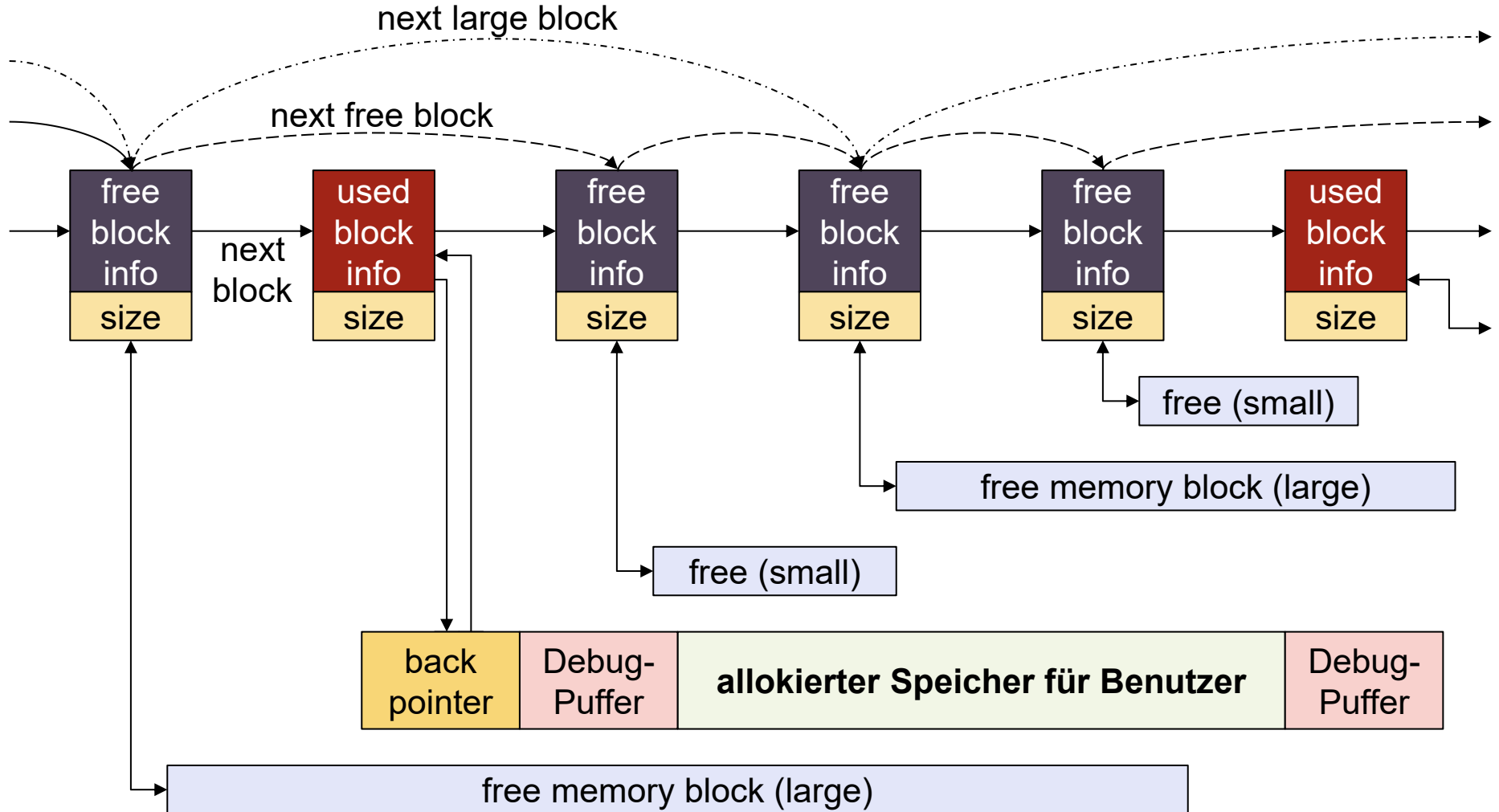
→ im Debug-Modus doppelt so viel

Datenstrukturen als Begründung:

- separate Verwaltungsstruktur mit Knoten für freie und belegte Speicherblöcke
 - Zeiger von einem Knoten zum nächsten Knoten
 - Hilfszeiger zum schnellen Finden passender Blöcke
 - Größe der Speicherblöcke zu den jeweiligen Knoten
 - Rückverweis vom Speicherblock zum Knoten
- Tricks (z. B. bucket heaps für kleine Allokationen) helfen bedingt
- Speicherfragmentierung durch viele kleine Allokationen
- im Debug-Modus: Puffer zum Erkennen von out-of-bounds-Schreibzugriffen

Naive Speicherallokation (6)

Beispiel Speicherverbrauch new



Allokatoren (1)

Grundüberlegungen

Ziel:

vermeide Overhead der Speicherverwaltung für spezielle Anlässe

Beispiele:

- keine Threadsicherheit nötig
- Objekte haben alle gleicher Größe
- Objekte werden immer alle zusammen gelöscht

Rahmenbedingung:

erlaube Weiterverwendung der STL-Container zur Redundanzvermeidung

Ansatz:

- zerlege Vorgang "Allokieren und Initialisieren" in "Allokieren" und "Initialisieren"
- zerlege Vorgang "Deallokieren und Deinitialisieren" in "Deallokieren" und "Deinitialisieren"
- erlaube benutzerdefinierte Operationen "Allokieren" und "Deallokieren"
- ein Objekt, das Speicher für andere Objekte allokiert und deallokiert, heißt **Allokator**

Allokatoren (2)

Umsetzung

- STL-Container haben zusätzlichen Templateparameter `A` für einen Allokator-Typ
- STL-Container-Instanz beinhaltet einen Member `allocator_` **fast** vom Typ `A`
- Speicherallokation und -deallokation wird an Funktionen von `allocator_` delegiert
- Initialisierung läuft über `placement new` und manuellen Destruktor-Aufruf

Beispiel:

```
template<class T, class A = std::allocator<T>> class list
{
    typename A::rebind<node<T>>::other allocator_;    // wir wollen node<T>, kein T!

    void insert(const T& value)
    {
        /* ... */
        node<T>* n = allocator_.allocate(1);          // Allokiere 1 node mit Allokator
        new (n) node<T>(value);                       // Initialisieren mit placement new
    }
};
```

Allokatoren (3)

Standard-Allokator:

- gibt man keinen Allokator-Typ `A` an, verwenden STL-Container `std::allocator<T>`
- der Standard-Allokator arbeitet intern z. B. mit `malloc()` / `free()` oder `new` / `delete`

eigene Allokatoren:

- Allokator-Typ muss bestimmte Voraussetzungen einhalten

→ ***Allokator-Konzept***

Voraussetzungen (Auszug!):

- `A::pointer` → typedef für Zeiger auf allokierten Typen
- `A::pointer a.allocate(n)` → allokieren mit Allokator-Instanz `a` Platz für `n` Objekte vom Typ `T`
- `a.deallocate(p, n)` → deallokiere Platz für `n` Objekte an `p` aus früherem `allocate(n)`
→ `n` muss bei `allocate()` und `deallocate()` übereinstimmen
- `A::template rebind<U>::other` → Vorlage `rebind` mit innerem typedef `other`
→ liefert Allokatortyp, der `U` statt `T` allokiert

Allokatoren (4)

Zustand als Teil vom Allokator-Konzept

bis C++11: **stateless allocators**

- Allokatoren dürfen keinen Zustand (= Membervariablen) haben
Beispiel: Allokator nur mit `static`-Variablen und `static`-Funktionen

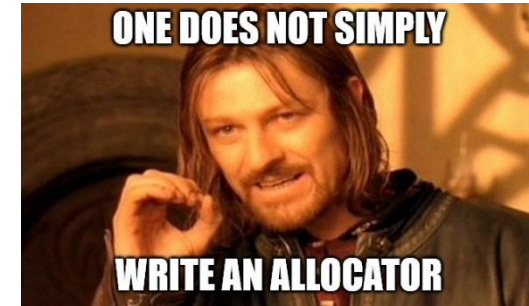
ab C++11: **stateful allocators**

- Allokatoren dürfen Zustand besitzen
Beispiel: Member-Array fester Größe als Speicherpool
- problematische Fragestellungen
Beispiel: Was passiert bei Container-Kopie mit dem Allokator?
- nicht-triviale Implementierung
Beispiel: `A::propagate_on_container_move_assignment` muss typedef für `std::true_type` sein, falls der allokierte Speicher beim Move des Containers mitverschoben werden kann, sonst `std::false_type`
→ verändert Verhalten von Move-Operationen auf dem Container

Allokatoren (5)

Eigene Allokatoren in der Praxis

- entwerfen korrekter Allokatoren ist nicht trivial
- Standard für Allokatoren wird nicht immer eingehalten
- speziellen Anwendungen genügt oft nicht nur ein neuer Allokator, sondern sie treffen auch sonst Grenzen der STL-Container



[New Line Cinema / WingNut]

→ oft ist es sinnvoller, gleich einen neuen Container zu schreiben!

wichtiger Seiteneffekt eigener Allokatoren:

Ausnahmesicherheit einiger Container-Methoden hängt vom Allokator-Typen ab!

Beispiel:

```
std::vector<T, A<T>>::vector() noexcept(noexcept(A<T>::A()))
```

→ Default-Konstruktor `vector` ruft Default-Konstruktor von Allokator `A` auf

→ `vector()` ist nur `noexcept`, wenn auch `A()` ein `noexcept` ist

Annahme dieser VL: Es wird nur der Standard-Allokator verwendet!

5 – Container

- Begriffe und Eigenschaften
- Iteratoren und Allokatoren
- **STL-Container**
- Member von STL-Containern

STL-Container (1)

Sequentielle Container

Eigenschaften:

- jedes Element an beliebig-wählbarer Position
- beliebig viele Duplikate an beliebiger Stelle erlaubt

E1	E2	E3	E4
17	32	17	22

- wenig Anforderungen an den inneren Datentyp
- vollständiges Durchgehen des Containers eher schnell
- andere Operationen (z. B. Suche) tendentiell langsam

Varianten:

- `array` → Wrapper um statisch dimensioniertes C-Array (C++11)
- `vector` → Wrapper um dynamisch allokiertes C-Array

- `forward_list` → einzeln verknüpfte Liste (C++11)
- `list` → doppelt verknüpfte Liste

- `deque` → Liste aus dynamisch allokierten C-Arrays

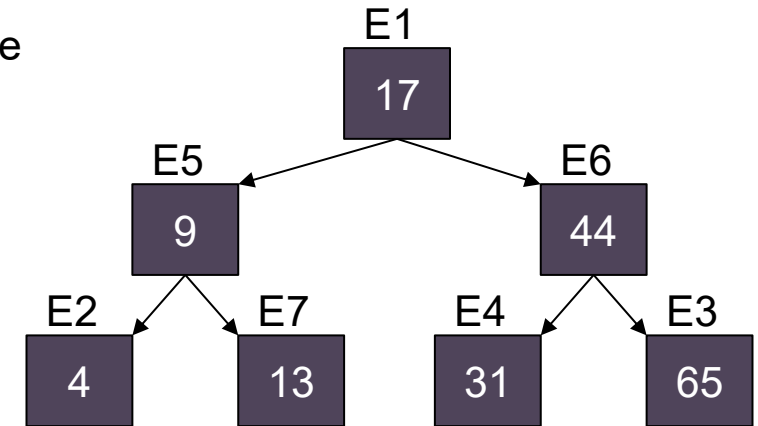
STL-Container (2)

Sortierende Container

(im Standard: *assoziative Container!*)

Eigenschaften:

- pro Vorlageninstanz eine wählbare Sortier-Reihenfolge
- Duplikate nur bei speziellen Containern möglich
- Duplikate alle an der selben Stelle im Container
- hohe Anforderungen an den inneren Datentyp
- alle Operationen tendentiell eher schnell
- eher großer Speicherplatzverbrauch



Varianten:

- `set` → eine sortierte Folge von eindeutigen Schlüssel-Elementen
- `map` → ein `set`, das jedem Schlüssel einen Wert zuordnet
- `multiset` → `set`, aber Schlüsselduplikate erlaubt
- `multimap` → `map`, aber Schlüsselduplikate (je mit eigenem Wert) erlaubt

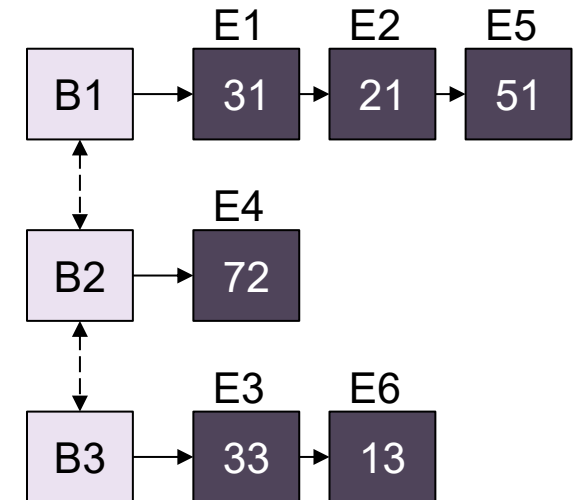
STL-Container (3)

Unsortierte Container

(seit C++11)

Eigenschaften:

- pro Vorlageninstanz eine Hashfunktion zur Kandidatenreduktion
- beliebige Umsortierung von Elementen durch den Container
- Duplikate nur bei speziellen Containern möglich
- Duplikate alle an der selben Stelle im Container
- mittlere Anforderungen an den inneren Datentyp
- alle Operationen tendentiell eher schnell bis sehr schnell
- eher großer Speicherplatzverbrauch



Varianten:

- `unordered_set` → eine unsortierte Folge von eindeutigen Schlüssel-Elementen
- `unordered_map` → ein `unordered_set`, das jedem Schlüssel einen Wert zuordnet
- `unordered_multiset` → `unordered_set`, aber Schlüsselduplikate erlaubt
- `unordered_multimap` → `unordered_map`, aber Schlüsselduplikate (je mit eigenem Wert)

STL-Container (4)

Container-Adapter

Eigenschaften:

- Schnittstelle, die nach außen speziellen Containertyp nachbildet
- Datenhaltung in Instanz eines wählbaren, inneren Containertyps
- Anforderungen und Operationen je nach innerem Containertyp
- spezielle Zusatzeigenschaften je nach Adapter-Typ

Varianten:

- `stack` → LIFO: Einfügen und Löschen immer an hinterster Stelle
- `queue` → FIFO: Einfügen vorne, Löschen hinten
- `priority_queue` → schnelles Entfernen von bestem Element bezüglich Sortierkriterium



STL-Container (5)

Grundlegende Struktur von Containern

Beobachtung:

Alle STL-Container folgen einem weitgehend einheitlichen Aufbau!
(z. B. bezüglich Anordnung und Benennung von Members)

Struktureinheiten:

- (1) Typexporte
- (2) Konstruktoren und Destruktoren
- (3) Zuweisungen
- (4) Iteratoren und Elementzugriff
- (5) Größe und Kapazität
- (6) Einfügen von Container-Inhalten
- (7) Löschen von Container-Inhalten
- (8) Hilfsfunktionen
- (9) globale Vergleichsoperatoren

Member types

Member type	Definition
value_type	T
allocator_type	Allocator
size_type	Unsigned integer type (usually <code>std::size_t</code>)
difference_type	Signed integer type (usually <code>std::ptrdiff_t</code>)
reference	<code>value_type&</code>
const_reference	<code>const value_type&</code>
pointer	<code>Allocator::pointer</code> (until C++11) <code>std::allocator_traits<Allocator>::pointer</code> (since C++11)
const_pointer	<code>Allocator::const_pointer</code> (until C++11) <code>std::allocator_traits<Allocator>::const_pointer</code> (since C++11)
iterator	<code>LegacyBidirectionalIterator</code> to <code>value_type</code>
const_iterator	<code>LegacyBidirectionalIterator</code> to <code>const value_type</code>
reverse_iterator	<code>std::reverse_iterator<iterator></code>
const_reverse_iterator	<code>std::reverse_iterator<const_iterator></code>

Member functions

<code>(constructor)</code>	constructs the list (public member function)
<code>(destructor)</code>	destructs the list (public member function)
<code>operator=</code>	assigns values to the container (public member function)
<code>assign</code>	assigns values to the container (public member function)
<code>get_allocator()</code>	returns the associated allocator

<https://en.cppreference.com>

→ Diese Struktur wird in der Regel (z. B. in Dokumentation) beibehalten!

5 – Container

- Begriffe und Eigenschaften
- Iteratoren und Allokatoren
- STL-Container
- Member von STL-Containern

Member von STL-Containern (1)

Typexporte

- `C<T>::value_type` → typedef für das τ des Containers
- `C<T>::reference` → typedef für $\tau\&$ und `const T&`
• `C<T>::const_reference`
- `C<T>::pointer` → typedef für τ^* und `const T^*`
• `C<T>::const_pointer`
- `C<T>::size_type` → typedef für `size_t` und `ptrdiff_t`
• `C<T>::difference_type`
- `C<T>::iterator` → typedef für Iteratoren, die bei ++ vorwärts gehen
• `C<T>::const_iterator`
- `C<T>::reverse_iterator` → s. o., aber bei ++ rückwärts
• `C<T>::const_reverse_iterator`



Member von STL-Containern (2)

Konstruktoren und Destruktoren

- `C<T>::C()` → Default-Konstruktor, leerer Container
→ dummerweise nicht bei allen Containern noexcept!
- `C<T>::C(const C& c)` → Copy-Konstruktor
- `C<T>::C(C&& c)` → Move-Konstruktor
→ dummerweise nicht bei allen Containern noexcept!
- `C<T>::C(size_t count)` → mit count default-konstruierten Elementen
- `C<T>::C(size_t count, const T& v)` → mit count aus v copy-konstruierten Elementen
- `C<T>::C(std::initializer_list<T>& l)` → mit Elementen aus { wert, wert, wert }-Liste
- `C<T>::~~C() noexcept` → Destruktor, gibt Speicher frei
→ invalidiert Referenzen und Iteratoren auf Inhalte
→ bei STL-Containern niemals virtual!

Anmerkung: Konstruktoren von Containern sind stark ausnahmesicher, wenn beteiligte Aufrufe auf τ ebenfalls stark ausnahmesicher sind!

Member von STL-Containern (3)

Zuweisungen

- `C& C<T>::operator =(const C& o)`
→ Copy-Zuweisungsoperator
- `C& C<T>::operator =(C&& o) noexcept`
→ Move-Zuweisungsoperator
→ Der hier ist netterweise immer `noexcept`!
- `C& C<T>::operator =(std::initializer_list<T>& l)`
→ Zuweisung einer { wert, wert, wert }-Liste
- `void C<T>::assign(size_t count, const T& v)`
- `template <typename I> void C<T>::assign(I first, I last)`
- `void C<T>::assign(std::initializer_list<T>& l)`
→ Zuweisung von Werten aus einem Bereich oder einer { }-Liste als Funktion

Anmerkung: Zuweisungen von Containern sind stark ausnahmesicher, wenn beteiligte Aufrufe auf τ ebenfalls stark ausnahmesicher sind!

Member von STL-Containern (5)

Iteratoren und Element-Zugriff

- `T& C<T>::front() noexcept` sowie `const T& C<T>::front() const noexcept`
- `T& C<T>::back() noexcept` sowie `const T& C<T>::back() const noexcept`
 - Zugriff aufs erste bzw. letzte Element
 - undefiniertes Verhalten bei leeren Containern
- `C::iterator C<T>::begin() noexcept`
- `C::const_iterator C<T>::begin() const noexcept`
- `C::iterator C<T>::end() noexcept`
- `C::const_iterator C<T>::end() const noexcept`
 - Iteratoren auf erstes Element und eins-nach-dem-letzten-Element
 - propagiere `const` vom Container auf den Iterator
 - analog: `rbegin()`, `rend()` für letztes Element und eins-nach-dem-ersten-Element
 - bei Bedarf: `cbegin()`, `cend()`, `crbegin()`, `crend()` geben immer `const_iterator` zurück

Anmerkung: `at(size_t index)` und `operator [ ](size_t index)` gibt es nur für `std::vector<T>`, nicht einmal für `std::list<T>`!

Member von STL-Containern (6)

Größe und Kapazität

- `bool C<T>::empty() noexcept`
→ `true`, wenn der Container kein einziges Element hat, sonst `false`
- `int C<T>::size() noexcept`
→ Zahl der Elemente in dem Container
- `int C<T>::max_size() noexcept`
→ Maximalzahl erlaubter Elemente in dem Container
→ 32-bit-Systeme: bestenfalls 4 GB / `sizeof(T)`
→ 64-bit-Systeme: üblicherweise ein unrealistisch großer Wert

Anmerkung: Direkte Veränderung der Größe und eine separate Kapazität gibt es nur bei wenigen Containern (z. B. `std::vector`)!

Member von STL-Containern (7)

Einfügen von Container-Inhalten

- `void C<T>::push_front(const T& value)`
- `void C<T>::push_front(T&& value)`
- `void C<T>::push_back(const T& value)`
- `void C<T>::push_back(T&& value)`
 - Einfügen am Anfang oder am Ende von sequentiellen Containern
- `C<T>::iterator C<T>::insert(/* iterator pos, */ const T& value)`
- `C<T>::iterator C<T>::insert(/* iterator pos, */ T&& value)`
- `template <typename I> C<T>::iterator C<T>::insert(/* iterator pos, */ I begin, I end)`
- `C<T>::iterator C<T>::insert(/* iterator pos, */ std::initializer_list<T>& l)`
 - Einfügen von einem oder mehreren Werten, optional per Verschieben
 - Position fürs Einfügen ist nur bei sequentiellen Containern möglich und nötig

Anmerkung: Einfügen in Containern ist stark ausnahmesicher, wenn beteiligte Aufrufe auf τ (z. B. `operator <=>` bei `std::set`) ebenfalls stark ausnahmesicher sind!

Member von STL-Containern (8)

Entfernen und Löschen von Container-Inhalten

- `void C<T>::pop_front()`
- `void C<T>::pop_back()`
 - Löschen vom ersten oder letzten Element bei sequentielllem Container
 - undefiniertes Verhalten bei leeren Containern
- `C<T>::iterator C<T>::erase(iterator pos)`
- `C<T>::iterator C<T>::erase(iterator first, iterator last)`
 - Löschen von einem oder mehreren Elementen
 - es gibt auch Überladungen für `const_iterator` als Parameter
 - Invalidierung von Iteratoren hängt stark vom beteiligten Container ab
- `void C<T>::clear() noexcept`
 - löscht alle Elemente im Container

Anmerkung: Löschen in Containern ist `noexcept`, wenn beteiligte Aufrufe auf `T` (z. B. `operator <=>` bei `std::set`) ebenfalls `noexcept` sind!

Member von STL-Containern (9)

Hilfsfunktionen

- `void C<T>::swap(C<T>& other) noexcept`
 - tauscht Inhalte von Containern aus
 - es wird nicht kopiert – es werden nur Datenzeiger getauscht!
- `void C<T>::merge(C<T>& other) noexcept`
- `void C<T>::merge(C<T>&& other) noexcept`
 - nur bei knotenbasierten Containern (z. B. list, set, map)
 - übernimm alle Elemente aus anderem Container, ohne zu kopieren
- `bool C<T>::contains(const T& value) const`
- `C<T>::iterator C<T>::find(const T& value)`
- `C<T>::const_iterator C<T>::find(const T& value) const`
 - suche nach Element mit gegebenem Wert im Container
 - nicht für sequentielle Container verfügbar

Member von STL-Containern (4)

Globale Vergleichsoperatoren

- `bool operator ==(const C<T>& lhs, const C<T>& rhs)`
 - Container sind gleich, wenn alle Elemente gleich und in gleicher Reihenfolge
 - Elemente sind gleich, wenn `T::operator ==(const T& other)` ein `true` zurückgibt
- `/* comparison result */ operator <=>(const C<T>& lhs, const C<T>& rhs)` (seit C++20)
 - `<=`, `>=`, `<`, `>`, `!=` werden automatisch erzeugt
 - Vergleich erfolgt lexikographisch, der erste vorgefundene Unterschied von erstem zu letztem Element im Container bestimmt das Ergebnis
- der Rückgabotyp von `operator <=>` hängt von `T` ab:
 - falls `T` selbst einen `operator <=>` hat, wird dessen Rückgabotyp verwendet
 - sonst wird `T::operator <` genutzt und der Ergebnistyp ist `std::weak_ordering`

Anmerkung: Vergleiche von Containern sind `noexcept`, wenn beteiligte Aufrufe auf `T` ebenfalls `noexcept` sind!

Zusammenfassung

Grundlagen

- Container als Sammlung von Elementen
- vielseitige Klassifizierung (z. B. Speicherbedarf, Laufzeit von Algorithmen)
- jeder Containertyp stellt **Forderungen** an Typ der enthaltenen Elemente
- Iteratoren und Allokatoren abstrahieren Container-Operationen

Wichtige Arten von Containern

- `array`, `vector`, `list`
- (geordneter) `set` und `map`
- `unordered_set` und `unordered_map`
- weitere, eher selten gebrauchte Container

Funktionen von Containern

- Konstruieren: Default-, Copy-, Move-Konstruktor
- Einfügen und Löschen von Elementen
- Zugriff auf und Iterieren von Elementen